

Liverpool FC Talent Pathway: A Short Reflection

UCD Professional Academy, Module 2 Python Assignment

Sean Philips

September 2026

What the app is for, and who it is for

The Liverpool FC Talent Pathway is a small Flask website built around a simple idea: what would it feel like to apply for a trial at a football club and find out, on the spot, what the club thinks you are worth?

The person I had in mind while building it is a young football fan, perhaps a teenager or a casual player, who wants a bit of fun rather than a serious tool. They land on the site, read a little about the club and its squad, fill in a short profile about themselves, and are given a rating out of 100. That rating then unlocks one or more contract offers, and they can pick one and “sign” it. The whole journey takes a couple of minutes and ends with a congratulations page.

I chose this idea because it gave me a natural reason to use every part of Flask the module covered. There is static content to read, a form to submit, calculations to run on the input, and a result that changes depending on what the user typed. It also kept me interested, which mattered when I hit the harder parts.

The routes, and where the app actually does something

There are six routes. Two of them are mostly there to be read, and four of them do real work.

The home page (/) introduces the club, and the squad page (/squad) lists the current players alongside which shirt numbers are still free. Both pull their content from a data module rather than having it typed straight into the templates, so changing a player or a fact means editing one Python list instead of hunting through HTML.

The trials page (/trials) is where the app comes alive. On a GET request it simply shows the form. On a POST it hands the submitted values to a validation function, which checks things like whether the age is a sensible number and whether the chosen position is one the club actually recognises. If anything is wrong, the same form is shown again with the user’s values kept in place and a clear message beside each problem. I spent more time on this than I expected, because a form that wipes everything you typed when you make one mistake is genuinely annoying to use.

If the profile passes, the user is sent to the offers page (/offers). This is where the rating is calculated. It starts from a base score and adjusts it for age (players in their mid twenties score best), how many days a week they train, and whether they are left footed,

right footed, or comfortable on both. The number is clamped so it always lands between 1 and 99. From that rating the app estimates a market value and a weekly wage, and filters a list of possible contracts so that only the ones the rating is good enough for are shown. A weaker profile might see a youth contract only, while a strong one sees the first team deal.

The last two routes handle signing. Posting to `/contract` shows the chosen offer for confirmation, and posting to `/contract/confirm` shows the final congratulations. One thing I am quietly proud of here is that the app never trusts the offer the browser sends back. It recalculates the rating, rebuilds the list of offers the user earned, and only accepts a choice that is on that list. I tested this by editing the hidden form field by hand on the live site, and the app refused the offer I had not earned. It was a small lesson in why you validate on the server, not just in the form.

How the templates are organised

All six pages extend a single base template. The base holds the HTML skeleton, the shared stylesheet link, the navigation bar and the footer, and leaves a content block for each page to fill. This meant that when I changed the navigation late on, I changed it once and every page picked it up.

Each route has its own template, named after the route so it is easy to find. Templates that show data loop over lists passed in from the view, and the form template uses conditionals to show error messages and repopulate fields. I tried to keep logic in the templates to a minimum: the templates decide how things look, and the Python decides what things are. When I caught myself writing a calculation in Jinja, I moved it into the ratings module instead.

The styling lives in one CSS file rather than being scattered inline, and the same layout works on a phone, which I checked because that is where most of my imagined users would be.

Looking back

The part of this assignment that taught me most was not the Flask syntax but the shape of the project: splitting routes, data, validation and calculations into separate files made every later change easier. If I did it again I would write the validation tests earlier, and I would probably give the rating formula a couple more inputs so the results felt less predictable. But as a first full web app, deployed and working at a public address, I am pleased with how it turned out.